

Browser APIs & Patterns

JavaScript browser-এ শুধু language হিসেবে কাজ করে না। Browser নিজেও JavaScript-এর জন্য অনেক powerful APIs দেয়।

এই chapter-এ আমরা এমন কিছু API ও pattern শিখব যগুলো frontend development-এর foundation:

- Event Delegation
- Fetch API
- Local Storage
- Session Storage
- History API
- URL
- URLSearchParams

React/Next.js-এর মতো framework ব্যবহার করলেও এগুলোর underlying concepts জানা খুব important।

1. Browser API কী?

Browser JavaScript runtime-এর সাথে বিভিন্ন built-in API দেয়।

যেমন:

```
fetch();  
localStorage;  
sessionStorage;  
history;  
location;  
URL;
```

এগুলো JavaScript language-এর core feature না।

বরং:

```
JavaScript  
+  
Browser APIs  
↓  
Web Application
```

উদাহরণ:

```
const data = await fetch("/api/users");
```

`fetch()` JavaScript-এর syntax না; এটি browser-provided Web API।

2. Event Delegation Pattern

Event delegation হলো এমন একটি pattern যেখানে অনেক child element-এর জন্য আলাদা event listener না দিয়ে তাদের common parent-এর উপর একটি listener বসানো হয়।

এটি মূলত **event bubbling** ব্যবহার করে।

3. Without Event Delegation

ধরো:

```
<ul>
  <li>Apple</li>
  <li>Banana</li>
  <li>Orange</li>
</ul>
```

প্রতিটি `li`-তে listener:

```
const items = document.querySelectorAll("li");

items.forEach(item => {
  item.addEventListener("click", () => {
    console.log(item.textContent);
  });
});
```

এখানে প্রতিটি element-এর জন্য listener তৈরি হচ্ছে।

4. Event Delegation দিয়ে

Parent-এর উপর listener:

```
const list = document.querySelector("ul");

list.addEventListener("click", event => {
  if (event.target.matches("li")) {
    console.log(event.target.textContent);
  }
});
```

এখানে:

```
ul
├─ li
├─ li
└─ li
```

শুধু `ul` -এর listener আছে।

5. কেন Event Delegation কাজ করে?

কারণ event সাধারণত child থেকে parent-এর দিকে bubble করে।

```
Click
  ↓
li
  ↓
ul
  ↓
body
  ↓
document
```

তাই `ul` event ধরতে পারে।

6. `event.target` vs `event.currentTarget`

Event delegation বুঝতে এটি খুব important।

```
list.addEventListener("click", event => {
  console.log(event.target);
});
```

```
console.log(event.currentTarget);
});
```

`event.target`

যে element-এ actual event ঘটেছে।

`event.currentTarget`

যে element-এর উপর listener বসানো হয়েছে।

উদাহরণ:

```
<ul>           ← currentTarget
  <li>          ← target
    <button>    ← target হতে পারে
  </li>
</ul>
```

7. `closest()` with Event Delegation

Real-world delegation-এ `closest()` খুব useful।

HTML:

```
<ul id="user-list">
  <li class="user">
    <button class="delete-btn">Delete</button>
  </li>
</ul>
```

JavaScript:

```
const list = document.querySelector("#user-list");

list.addEventListener("click", event => {
  const button = event.target.closest(".delete-btn");

  if (!button) return;

  const user = button.closest(".user");
```

```
    console.log(user);  
  });
```

এটি nested elements থাকলেও robust ।

8. Dynamic Elements-এর জন্য Event Delegation

Event delegation-এর সবচেয়ে বড় সুবিধাগুলোর একটি হলো dynamically added elements-এর সাথেও কাজ করে ।

```
const list = document.querySelector("#list");  
  
list.addEventListener("click", event => {  
  if (event.target.matches(".item")) {  
    console.log("Clicked:", event.target.textContent);  
  }  
});
```

পরে:

```
const item = document.createElement("li");  
  
item.className = "item";  
item.textContent = "New Item";  
  
list.append(item);
```

নতুন item-এর জন্য আলাদা listener লাগবে না ।

9. কখন Event Delegation ব্যবহার করবে?

Useful যখন:

- অনেক similar child element আছে
- Dynamic elements তৈরি হচ্ছে
- List/table/grid আছে
- অনেক listener avoid করতে চাও

উদাহরণ:

```
Todo list
User table
Product grid
Menu
Dropdown
Comment list
```

10. Event Delegation-এর Limitation

সব event delegation-এর জন্য equally suitable না।

বিশেষ করে event-এর bubbling behavior বুঝতে হবে।

কিছু event traditional bubbling pattern follow করে না বা special behavior থাকতে পারে।

তাই blindly delegation ব্যবহার না করে event behavior বুঝে ব্যবহার করবে।

11. Fetch API

`fetch()` browser থেকে HTTP request করার modern API।

Basic:

```
const response = await fetch("/api/users");
```

GET request:

```
const response = await fetch("/api/users");

const data = await response.json();

console.log(data);
```

12. Fetch-এর Mental Model

Fetch-এর flow:

```
fetch(url)
  ↓
Promise<Response>
  ↓
Response
  ↓
response.json()
  ↓
Promise<data>
  ↓
JavaScript object/array
```

Important:

`fetch()` সরাসরি JSON data return করে না।

এটি প্রথমে `Response` object return করে।

13. Basic Fetch

```
async function getUsers() {
  const response = await fetch("/api/users");

  const data = await response.json();

  return data;
}
```

14. HTTP Status Check

এটি Fetch API-এর সবচেয়ে important conceptগুলোর একটি।

অনেকেই ভাবে:

```
try {
  const response = await fetch("/api/users");
} catch (error) {
```

```
    console.log(error);  
  }
```

404 বা 500 হলেই `catch` চলবে।

সাধারণভাবে এমন নয়।

`fetch()` network-level failure-এর ক্ষেত্রে reject করে; HTTP error status যেমন 404/500 সাধারণত `Response` হিসেবেই পাওয়া যায়।

তাই:

```
if (!response.ok) {  
  throw new Error(`HTTP error: ${response.status}`);  
}
```

15. Proper Fetch Error Handling

```
async function getUsers() {  
  try {  
    const response = await fetch("/api/users");  
  
    if (!response.ok) {  
      throw new Error(  
        `Request failed: ${response.status}`  
      );  
    }  
  
    return await response.json();  
  } catch (error) {  
    console.error("Failed to fetch users:", error);  
    throw error;  
  }  
}
```

16. Response.ok

```
response.ok
```

সাধারণভাবে successful HTTP status range indicate করে।

Example:

```
if (response.ok) {  
  console.log("Success");  
}
```

17. Useful Response Properties

```
response.status;  
response.statusText;  
response.ok;  
response.url;  
response.headers;
```

Example:

```
console.log(response.status);  
console.log(response.ok);
```

18. Response Body Methods

Response body parse করার জন্য:

```
response.json();
```

আরও methods আছে:

```
response.text();  
response.blob();
```

```
response.arrayBuffer();  
response.formData();
```

JSON

```
const data = await response.json();
```

Text

```
const text = await response.text();
```

Blob

Images/files-এর মতো binary data handle করতে useful।

```
const blob = await response.blob();
```

19. Response Body একবারই Consume করা যায়

সাধারণভাবে একই response body বারবার consume করা যায় না।

```
const response = await fetch("/api/data");  
  
const data = await response.json();  
  
const text = await response.text();
```

এটি সাধারণভাবে কাজ করবে না, কারণ body already consumed।

20. Request Method

GET:

```
fetch("/api/users");
```

POST:

```
fetch("/api/users", {  
  method: "POST"  
});
```

PUT:

```
fetch("/api/users/1", {  
  method: "PUT"  
});
```

PATCH:

```
fetch("/api/users/1", {  
  method: "PATCH"  
});
```

DELETE:

```
fetch("/api/users/1", {  
  method: "DELETE"  
});
```

21. POST JSON Request

```
const response = await fetch("/api/users", {  
  method: "POST",  
  
  headers: {  
    "Content-Type": "application/json"  
  },  
  
  body: JSON.stringify({  
    name: "Sharlar",  
    age: 31  
  })  
});
```

Flow:

```
JavaScript object
  ↓
JSON.stringify()
  ↓
HTTP body
  ↓
Server
```

22. Fetch Headers

Request headers:

```
fetch("/api/users", {
  headers: {
    Authorization: "Bearer token",
    "Content-Type": "application/json"
  }
});
```

Response headers:

```
const contentType =
  response.headers.get("content-type");

console.log(contentType);
```

23. Fetch with Query Parameters

```
const url = new URL("/api/users", window.location.origin);

url.searchParams.set("page", "2");
url.searchParams.set("limit", "20");

const response = await fetch(url);
```

এখানে URL হবে conceptually:

```
/api/users?page=2&limit=20
```

24. Fetch + AbortController

Long-running request cancel করার জন্য `AbortController` useful।

```
const controller = new AbortController();

const response = await fetch("/api/users", {
  signal: controller.signal
});
```

Cancel:

```
controller.abort();
```

এটি useful:

- Search requests
- Component cleanup
- Timeout
- User navigation
- Avoiding stale requests

25. Local Storage

`localStorage` browser-এ data persist করার জন্য ব্যবহৃত হয়।

```
localStorage.setItem("theme", "dark");
```

Read:

```
const theme = localStorage.getItem("theme");

console.log(theme);
```

Remove:

```
localStorage.removeItem("theme");
```

সব clear:

```
localStorage.clear();
```

26. Local Storage Persistence

`localStorage` সাধারণত browser session-এর পরেও data রাখে।

```
Browser opened
  ↓
setItem()
  ↓
Browser closed
  ↓
Browser reopened
  ↓
Data remains
```

তাই এটি persistent client-side storage।

27. Local Storage Stores Strings

এটি খুব important।

```
localStorage.setItem("age", 31);
```

পরে:

```
const age = localStorage.getItem("age");
console.log(typeof age);
```

Output:

```
string
```

কারণ storage values string হিসেবে stored হয়।

28. Objects Store করতে JSON

এভাবে:

```
const user = {  
  name: "Sharlar",  
  age: 31  
};  
  
localStorage.setItem(  
  "user",  
  JSON.stringify(user)  
);
```

Read:

```
const storedUser = JSON.parse(  
  localStorage.getItem("user")  
);
```

29. Safe JSON Parsing

যদি key না থাকে:

```
localStorage.getItem("user");
```

return করতে পারে:

```
null
```

তাই:

```
const stored = localStorage.getItem("user");

const user = stored
  ? JSON.parse(stored)
  : null;
```

30. Local Storage Common Use Cases

Useful for:

- Theme preference
- UI preference
- Small client-side settings
- Non-sensitive persistence
- Recently selected filters
- Draft data

31. Local Storage Security Warning

`localStorage` sensitive secrets রাখার জন্য ideal না।

বিশেষ করে authentication token-এর ক্ষেত্রে XSS risk এবং application architecture বিবেচনা করতে হয়।

তাই blindly:

```
localStorage.setItem(
  "accessToken",
  token
);
```

করার আগে security model বুঝতে হবে।

32. Session Storage

`sessionStorage` API `localStorage`-এর মতো:

```
sessionStorage.setItem("step", "2");
```

Read:

```
sessionStorage.getItem("step");
```

Remove:

```
sessionStorage.removeItem("step");
```

Clear:

```
sessionStorage.clear();
```

33. Local Storage vs Session Storage

Feature	localStorage	sessionStorage
Persistence	Long-term	Current tab/session
Browser restart	Usually remains	Usually cleared with tab/session lifecycle
API	Same style	Same style
Stores	Strings	Strings
Scope	Origin	Origin + browsing tab
Capacity	Limited	Limited

34. sessionStorage Tab-Based Behavior

Conceptually:

```
Tab A
└─ sessionStorage
```

Tab B

└─ separate sessionStorage

তাই `sessionStorage` tab/session-specific state-এর জন্য useful ।

35. Storage API

দুটোর API প্রায় একই:

```
storage.setItem(key, value);  
storage.getItem(key);  
storage.removeItem(key);  
storage.clear();  
storage.key(index);  
storage.length;
```

Example:

```
console.log(localStorage.length);
```

36. Storage Events

একটি useful browser feature:

```
window.addEventListener("storage", event => {  
  console.log(event.key);  
  console.log(event.newValue);  
});
```

এটি অন্য document/context-এ storage পরিবর্তনের সাথে coordination-এর জন্য useful ।

37. Storage Limitations

`localStorage` এবং `sessionStorage`:

- synchronous

- string-based
- relatively small
- database-এর replacement না
- large data storage-এর জন্য suitable না
- sensitive data-এর জন্য blindly suitable না

Large structured client-side data-এর জন্য IndexedDB-এর মতো API বেশি appropriate হতে পারে।

38. History API

Browser history manipulate করার জন্য:

```
history.pushState();  
history.replaceState();
```

এটি SPA routing-এর underlying browser concept বোঝার জন্য important।

39. history.pushState()

নতুন history entry যোগ করে।

```
history.pushState(  
  { page: "profile" },  
  "",  
  "/profile"  
);
```

Concept:

```
/history  
↓  
/profile
```

Back করলে আগের history entry-তে যেতে পারবে।

40. replaceState()

Current history entry replace করে।

```
history.replaceState(  
  { page: "profile" },  
  "",  
  "/profile"  
);
```

এখানে নতুন history entry তৈরি হয় না।

Concept:

```
Current entry  
  ↓  
replace  
  ↓  
New URL
```

41. `pushState()` vs `replaceState()`

Method	New Entry	Back button
<code>pushState()</code>	Yes	Previous entry-তে যেতে পারে
<code>replaceState()</code>	No	Replaced entry-এর জন্য নতুন history step নেই

Mental model:

```
pushState  
= "Add another page"  
  
replaceState  
= "Change current page"
```

42. State Object

`pushState()` -এ state data রাখা যায়।

```
history.pushState(  
  {
```

```
    page: "products",
    filter: "cheap"
  },
  "",
  "/products"
);
```

পরে:

```
console.log(history.state);
```

43. popstate Event

Back/forward navigation detect করতে:

```
window.addEventListener("popstate", event => {
  console.log(event.state);
});
```

Flow:

```
Back / Forward
  ↓
popstate
  ↓
Read history state
  ↓
Update UI
```

44. History API এবং SPA

Traditional website:

```
Click link
  ↓
Server request
```

↓
New document

SPA:

```
Click
  ↓
JavaScript
  ↓
pushState()
  ↓
URL changes
  ↓
UI updates
```

React Router, Next.js এবং অন্যান্য routing systems-এর underlying browser concepts-এর মধ্যে History API গুরুত্বপূর্ণ।

Framework নিজের routing abstraction তৈরি করে, কিন্তু browser-level history behavior বোঝা থাকলে routing সহজে বুঝতে পারবে।

45. History API Does Not Automatically Render Pages

এটি important।

```
history.pushState(
  {},
  "",
  "/about"
);
```

এতে URL change হবে।

কিন্তু browser নিজে থেকে React component render করবে না।

তোমাকেই UI update করতে হবে অথবা framework/router সেটা করবে।

46. URL API

URL manipulate করার জন্য `URL` constructor খুব useful।

```
const url = new URL(  
  "https://example.com/products?page=2"  
);
```

তারপর:

```
console.log(url.hostname);  
console.log(url.pathname);  
console.log(url.search);
```

47. URL-এর Structure

একটি URL:

```
https://example.com:8080/products?page=2#reviews
```

এর অংশ:

```
https  
↓  
protocol  
  
example.com  
↓  
hostname  
  
8080  
↓  
port  
  
/products  
↓  
pathname  
  
?page=2  
↓
```

search/query

#reviews

↓

hash

48. URL Properties

```
const url = new URL(
  "https://example.com/products?page=2#reviews"
);

console.log(url.protocol);
console.log(url.hostname);
console.log(url.port);
console.log(url.pathname);
console.log(url.search);
console.log(url.hash);
```

49. URL Mutation

URL object mutable।

```
const url = new URL(
  "https://example.com/products"
);

url.pathname = "/users";

console.log(url.href);
```

50. URLSearchParams

Query parameters handle করার জন্য:

```
const params = new URLSearchParams(  
  "?page=2&limit=20"  
);
```

Read:

```
console.log(params.get("page"));
```

Output:

```
2
```

51. `set()`

```
params.set("page", "3");
```

এটি existing value replace করে অথবা নতুন parameter add করে।

52. `append()`

```
params.append("tag", "javascript");  
params.append("tag", "react");
```

Result conceptually:

```
tag=javascript&tag=react
```

Difference:

```
set()  
→ replace existing value  
  
append()  
→ add another value
```

53. has()

```
params.has("page");
```

Boolean return করে।

54. delete()

```
params.delete("page");
```

Parameter remove করবে।

55. getAll()

যদি একই key multiple times থাকে:

```
const params = new URLSearchParams(  
  "tag=js&tag=react"  
);  
  
console.log(params.getAll("tag"));
```

Output:

```
["js", "react"]
```

56. Iterating URLSearchParams

```
for (const [key, value] of params) {  
  console.log(key, value);  
}
```

অথবা:

```
params.forEach((value, key) => {  
  console.log(key, value);  
});
```

57. Building URLs Safely

Bad:

```
const url =  
  "/search?q=" + search + "&page=" + page;
```

Special characters থাকলে সমস্যা হতে পারে।

Better:

```
const params = new URLSearchParams();  
  
params.set("q", search);  
params.set("page", page);  
  
const url = `/search?${params}`;
```

58. URL + URLSearchParams Together

```
const url = new URL(  
  "https://example.com/search"  
);  
  
url.searchParams.set("q", "JavaScript");  
url.searchParams.set("page", "2");  
  
console.log(url.href);
```

এটি clean এবং reliable।

59. URL Encoding

Query parameter-এর মধ্যে special character থাকলে encoding দরকার।

```
const params = new URLSearchParams();

params.set("q", "hello world");

console.log(params.toString());
```

Browser appropriate encoding করবে।

তাই manually encoding logic লেখার বদলে `URLSearchParams` ব্যবহার করা safer।

60. Reading Current Page URL

Browser-এ:

```
console.log(window.location.href);
```

আর:

```
window.location.pathname;
window.location.search;
window.location.hash;
```

61. URLSearchParams from Current URL

```
const params = new URLSearchParams(
  window.location.search
);

const page = params.get("page");
```

যদি URL:

```
/products?page=2&sort=price
```

তাহলে:

```
params.get("page");  
// "2"  
  
params.get("sort");  
// "price"
```

62. Practical Example — Search Page

URL:

```
/products?search=laptop&page=2&sort=price
```

JavaScript:

```
const params = new URLSearchParams(  
  window.location.search  
);  
  
const search = params.get("search");  
const page = params.get("page");  
const sort = params.get("sort");  
  
console.log(search);  
console.log(page);  
console.log(sort);
```

63. Practical Example — Updating Filters

```
const url = new URL(window.location.href);  
  
url.searchParams.set("category", "laptop");  
url.searchParams.set("page", "1");
```

```
history.pushState(  
  {},  
  "",  
  url  
);
```

এখন:

```
URL  
↓  
Query params  
↓  
Application state  
↓  
Fetch data  
↓  
Render UI
```

এটি SPA application-এ খুব common pattern ।

64. Event Delegation + Dynamic UI

একটি practical example:

```
const list = document.querySelector("#products");  
  
list.addEventListener("click", event => {  
  const button = event.target.closest(  
    "[data-product-id]"  
  );  
  
  if (!button) return;  
  
  const id = button.dataset.productId;  
  
  console.log("Product:", id);  
});
```

HTML:

```
<button data-product-id="101">
  Add to cart
</button>
```

এখানে event delegation + `dataset` একসাথে কাজ করছে।

65. Fetch + URLSearchParams

```
async function searchProducts(query, page) {
  const params = new URLSearchParams();

  params.set("q", query);
  params.set("page", page);

  const response = await fetch(
    `/api/products?${params}`
  );

  if (!response.ok) {
    throw new Error(
      `Request failed: ${response.status}`
    );
  }

  return response.json();
}
```

এটি production frontend-এর খুব common pattern।

66. Local Storage + JSON

একটি theme preference:

```
function saveTheme(theme) {
  localStorage.setItem("theme", theme);
}

function getTheme() {
```

```
    return localStorage.getItem("theme");  
}
```

Object:

```
function saveUserPreferences(preferences) {  
    localStorage.setItem(  
        "preferences",  
        JSON.stringify(preferences)  
    );  
}  
  
function getUserPreferences() {  
    const value =  
        localStorage.getItem("preferences");  
  
    return value ? JSON.parse(value) : null;  
}
```

67. History + Fetch Pattern

একটি SPA search page-এর flow হতে পারে:

```
User changes filter  
    ↓  
Update URLSearchParams  
    ↓  
history.pushState()  
    ↓  
Read query params  
    ↓  
fetch()  
    ↓  
Update UI
```

Back button:

```
User clicks Back  
    ↓  
popstate  
    ↓
```

```
Read URL
  ↓
Fetch corresponding data
  ↓
Render UI
```

এই pattern SPA architecture বোঝার জন্য অত্যন্ত important ।

68. Browser APIs Relationship

এই chapter-এর APIs আলাদা আলাদা মনে হলেও বাস্তবে একসাথে কাজ করে ।

```
Event Delegation
  ↓
User interaction
  ↓
URLSearchParams
  ↓
History API
  ↓
Fetch API
  ↓
Response
  ↓
UI update

Local Storage
  ↓
Persist preferences
```

69. React-এর সাথে Connection

React application-এ এগুলো directly বা indirectly ব্যবহার হয় ।

Fetch

```
useEffect(() => {
  fetch("/api/users");
}, []);
```

Local Storage

```
localStorage.setItem("theme", "dark");
```

URLSearchParams

```
const params = new URLSearchParams(  
  window.location.search  
);
```

History

Routing libraries browser History API-এর উপর abstraction তৈরি করতে পারে।

70. Important React Warning

React application-এ:

```
localStorage.getItem(...)
```

বা:

```
window.location
```

সব জায়গায় blindly ব্যবহার করা যাবে না।

বিশেষ করে SSR environment যেমন Next.js-এ server-side execution-এর সময় `window` বা `localStorage` available নাও থাকতে পারে।

তাই browser-only APIs ব্যবহার করার সময় execution environment বুঝতে হবে।

71. Common Mistakes

Mistake 1 — Fetch 404 মানেই catch

ভুল ধারণা:

```
try {
  await fetch("/wrong-url");
} catch {
  console.log("404");
}
```

HTTP status explicitly check করে:

```
const response = await fetch("/wrong-url");

if (!response.ok) {
  throw new Error(`HTTP ${response.status}`);
}
```

Mistake 2 — localStorage-এ Object সরাসরি রাখা

```
localStorage.setItem("user", user);
```

এভাবে object meaningfulভাবে store হবে না।

Use:

```
localStorage.setItem(
  "user",
  JSON.stringify(user)
);
```

Mistake 3 — `pushState()` UI automatically update করবে ভাবা

```
history.pushState({}, "", "/profile");
```

এটি URL change করে।

UI update automatically হয় না।

Mistake 4 — `set()` এবং `append()` একই ভাবা

```
params.set("tag", "js");
```

একটি value replace করে।

```
params.append("tag", "react");
```

আরেকটি value যোগ করতে পারে।

Mistake 5 — Event Delegation-এ target ভুল বোঝা

```
event.target
```

এবং:

```
event.currentTarget
```

এক জিনিস না।

72. Best Practices

Event Delegation

- Common parent-এ listener বসানো
- `closest()` ব্যবহার করো যখন nested elements থাকে
- Dynamic UI-তে delegation consider করো

Fetch

- `response.ok` check করো
- JSON parse করার আগে response handle করো
- HTTP error এবং network error আলাদা করে ভাবে
- প্রয়োজন হলে `AbortController` ব্যবহার করো

Storage

- Small, non-sensitive data রাখো
- Object-এর জন্য JSON ব্যবহার করো

- Sensitive authentication data নিয়ে security model বুঝে সিদ্ধান্ত নাও

History

- `pushState()` = new navigation state
- `replaceState()` = current state replace
- Back/Forward handle করতে `popstate` বুঝো

URL

- URL manually concatenate করার বদলে `URL` / `URLSearchParams` ব্যবহার করো
 - Query parameter encoding নিজে manually করার প্রয়োজন কমাও
-

73. Practice — Easy

Task 1

একটি `<ul>` -এর জন্য event delegation implement করো।

Task 2

`fetch()` দিয়ে একটি API request করো এবং:

- status check করো
 - JSON parse করো
 - error handle করো
-

Task 3

`localStorage` -এ একটি theme save এবং read করো।

Task 4

`sessionStorage` -এ একটি form step save করো।

Task 5

এই URL থেকে `page` এবং `sort` বের করো:

```
/products?page=3&sort=price
```

74. Practice — Medium

Task 6

Dynamic todo list তৈরি করো যেখানে:

- parent-এর একটি click listener থাকবে
- dynamically added todo-ও কাজ করবে
- delete button `closest()` দিয়ে detect করবে

Task 7

এই function তৈরি করো:

```
buildSearchUrl({  
  query,  
  page,  
  sort  
});
```

যেটি `URLSearchParams` ব্যবহার করে URL return করবে।

Task 8

একটি SPA-like navigation তৈরি করো:

```
Home → Products → Profile
```

`pushState()` ব্যবহার করে।

তারপর Back button-এর জন্য `popstate` handle করো।

75. Practice — Advanced

Task 9

একটি search page তৈরি করে:

```
/products?q=laptop&page=2
```

Flow:

```
URL
↓
URLSearchParams
↓
fetch()
↓
Render products
```

Task 10

Filter change করলে:

```
Update query params
↓
pushState()
↓
fetch()
↓
render
```

এবং browser Back করলে:

```
popstate
↓
read URL
↓
fetch previous state
```

implement করো।

76. Interview Questions

Beginner

1. Event delegation কী?
2. Event bubbling কীভাবে delegation-এর সাথে related?
3. `event.target` এবং `event.currentTarget` -এর পার্থক্য কী?
4. `fetch()` কী return করে?
5. `response.json()` কী করে?
6. `localStorage` এবং `sessionStorage` কী?
7. `localStorage` -এ object কীভাবে রাখবে?
8. `pushState()` কী?
9. `replaceState()` কী?
10. `URLSearchParams` কেন ব্যবহার করা হয়?

Intermediate

1. `fetch()` কখন reject করে?
2. HTTP 404/500 handle করতে `response.ok` কেন check করতে হয়?
3. Event delegation dynamic elements-এর ক্ষেত্রে কেন useful?
4. `localStorage` কেন synchronous?
5. `localStorage` এবং `sessionStorage` -এর scope difference কী?
6. `pushState()` UI automatically update করে না কেন?
7. `popstate` কখন fire হয়?
8. URL এবং `URLSearchParams` -এর relationship কী?
9. `set()` এবং `append()` -এর পার্থক্য কী?
10. Query parameters manually concatenate করার সমস্যা কী?

Advanced

1. Event delegation-এর performance benefit কী?
2. `fetch()` -এর network error এবং HTTP error-এর পার্থক্য কী?
3. Response body কেন সাধারণভাবে একবার consume করা যায়?
4. SPA routing-এর সাথে History API কীভাবে কাজ করে?
5. URL state এবং application state কীভাবে synchronize করবে?
6. Browser storage-এর security limitations কী?
7. Next.js/SSR-এ `window` বা `localStorage` ব্যবহার করার সময় কী সমস্যা হতে পারে?
8. `AbortController` কীভাবে fetch cancellation-এর সাথে কাজ করে?

77. Quick Revision

Event Delegation



Parent listener

↓
Event bubbling
↓
event.target

fetch()
↓
Promise<Response>
↓
response.ok
↓
response.json()
↓
Data

localStorage
↓
Persistent
↓
String values

sessionStorage
↓
Tab/session scoped
↓
String values

pushState()
↓
New history entry

replaceState()
↓
Replace current entry

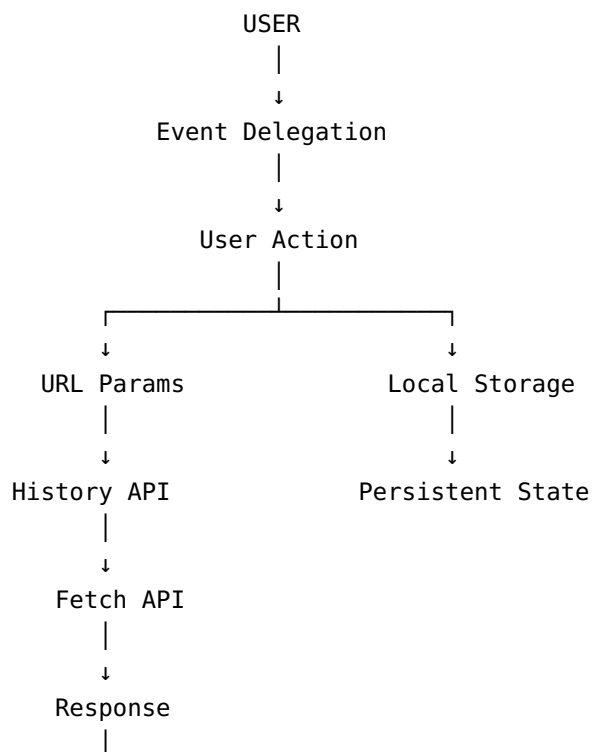
Back / Forward
↓
popstate

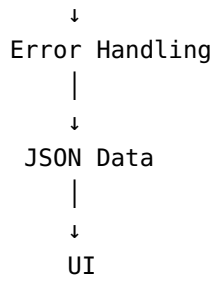
URL
↓
protocol
hostname
pathname
search
hash

URLSearchParams
↓
get()
set()
append()
has()
delete()
getAll()

78. Final Mental Model

পুরো chapter-টাকে একটি modern frontend application হিসেবে চিন্তা করো:





এই ৭টা concept একসাথে বুঝলে browser-side JavaScript-এর একটা বড় foundation তৈরি হয়ে যায়।

সবচেয়ে important:

Event Delegation
→ efficient event handling

Fetch
→ HTTP communication

localStorage
→ persistent browser storage

sessionStorage
→ session/tab storage

History API
→ browser navigation state

URL
→ URL parsing/manipulation

URLSearchParams
→ query-string management

Browser API শেখার মূল লক্ষ্য শুধু **API**-এর **method** মুখস্থ করা না; বরং এগুলো কীভাবে একসাথে একটি **real frontend application** তৈরি করে সেটা বোঝা।